Bomb Lab 实验报告

第一关：函数调用

这一关输入一个无符号整数。phase_01 调用 p1_compute，再将返回值与输入比较。关键指令如下：

```
p1_step2:  lea (%rdi,%rsi,2),%eax
           xor $0x3ebb,%eax
p1_step0:  xor $0xc25c,%esi
           lea (%rsi,%rdi,4),%eax
p1_step1:  lea (%rdi,%rdi,2),%eax
           add $0x469a,%esi
           xor %esi,%eax
```

下面用 c 表示各关的 64 位 challenge，u32(x) 表示取低 32 位，^ 表示按位异或。32 位运算使用 uint32_t，溢出时只保留低 32 位，右移按无符号数处理。

先看调用前的寄存器赋值，得到 L=u32(c)、H=u32(c>>32)、M=u32(c>>16)。其中 M 是右移后取低 32 位。

```
A = (H + 2*L) ^ 0x3ebb
B = 4*L + (M ^ 0xc25c)
answer = (3*B) ^ (A + 0x469a)
```

调用顺序为 p1_step2(H,L)、p1_step0(L,M)、p1_step1(B,A)，依次代入即可得到上面的结果。lea 在这里用于加法和倍数计算。

第二关：数组与循环

这一关需要输入六个整数，首项的检查指令为：

```
shr $0x2b,%rax
add %eax,%eax
and $0x1e,%eax
add $0x4,%eax
```

因此 a[0] = 2*((c>>43)&15)+4。循环从移位量 0x16=22 开始，每轮增加 3；lea (%rdi,%rdi,4),%edi 表示将前一项乘以 5。

```
add $0x1,%esi
lea (%rdi,%rdi,4),%edi
shr %cl,%rax
add $0x3,%ecx
and $0x1f,%eax
xor %esi,%eax
add %edi,%eax
```

恢复为：

```
a[0] = 2*((c>>43)&15)+4
for i = 1..5:
    fragment = (c>>(22+3*(i-1))) & 31
    a[i] = 5*a[i-1] + (fragment ^ (i+1))
```

第一次循环中 esi 先从 1 加到 2，因此公式中用 i+1。最后按顺序输出六项。

第三关：switch 与跳转表

这一关输入两个整数。第一个是 k=c&7，第二个由 x=u32(c>>32)^u32(c>>3) 经过对应分支计算得到。

```
402268: cmp $0x7,%eax
40226b: ja 4022b5
40226d: notrack jmp *0x405820(,%rax,8)
```

跳转表有 8 个表项，每项是 8 字节地址。读取表项，再跟踪跳板指令，得到下表：

k	跳板地址	最终函数	结果
0	0x402288	p3_case_0	3*x+0x1b0a
1	0x402290	p3_case_2	5*x+0x2fd
2	0x402298	p3_case_7	5*x+0x30ad9
3	0x4022a0	p3_case_4	(4*x)^0x2088
4	0x4022a8	p3_case_1	(x^0x60bc)+(x>>2)
5	0x4022b0	p3_case_3	(4*x)^0xbfeb
6	0x402278	p3_case_6	3*x+0xa7aa
7	0x402280	p3_case_5	(x^0x8275)+(x>>1)

代码用 switch(k) 计算对应分支，输出 k 和结果。

第四关：递归与返回值

入口提取 target=c&127、salt=u32(c>>7)^u32(c>>32)，以区间 [0,127] 调用 p4_walk。中点由 high-low 右移一位后加 low 得到。

```
402348: sub %esi,%edi
40234a: shr $1,%edi
40234c: add %esi,%edi
402350: je 4023a0
...
40236c: lea (%rax,%rax,2),%eax
40236f: xor %ebx,%eax
402376: xor $0x19,%eax
...
402394: xor $0x2,%eax
4023a0: lea 0xcb(%rdi),%eax
4023a6: xor %ecx,%eax
```

令 m=low+((high-low)>>1)，递归关系为：

```
F(t,l,h,s):
    m = l + ((h-l)>>1)
    if t == m: return (m+0xcb)^s
    if t < m:  return (3*F(t,l,m-1,s))^(m+s)^2
    otherwise: return (3*F(t,m+1,h,s))^(m+s)^0x19
```

口令是 F(target,0,127,salt)。先通过二分找到 target，再从内向外计算每层的返回值。jb 判断左分支，左右分支分别异或 2 和 0x19。target 在 0～127 内，区间每次缩小，最多 8 次调用就能找到。

第五关：结构体与链表

p5_traverse 从地址表 0x405b60 选择起点。节点内存读取为：

```
mov 0x405b60(,%rax,8),%rdx  # 起点地址
mov 0x8(%rdx),%esi          # tag
add 0xc(%rdx),%eax          # value
mov (%rdx),%rdx             # next
```

由这些偏移可以看出，每个节点有 8 字节 next 指针、4 字节 tag 和 4 字节 value，共 16 字节。各节点数据如下，代码中用数组索引表示 next：

索引	节点地址	next 索引	tag	value
0	0x405ba0	9	1399	15708
1	0x405bb0	4	100	15737
2	0x405bc0	10	1887	17130
3	0x405bd0	1	2689	49663
4	0x405be0	6	2883	42131
5	0x405bf0	0	1614	14186
6	0x405c00	7	2352	18482
7	0x405c10	5	611	31608
8	0x405c20	2	874	15671
9	0x405c30	空	2106	64991
10	0x405c40	11	258	47895
11	0x405c50	3	1122	33207

起点索引表为 [8,2,10,11,3,1,4,6]。在 heads[c&7] 处开始，访问 1+((c>>3)&3) 个节点：

```
v = u32(c>>6) ^ u32(c>>32) ^ 0x40b9
每访问一个节点 p:
    v = (3*v + p.value) ^ p.tag
    last_tag = p.tag
    p = p.next
输出 v, last_tag
```

起点先处理一次，再由 (c>>3)&3 决定后续次数，因此一共访问 1～4 个节点。输出累计值和最后一个节点的 tag。

第六关：函数指针

p6_dispatch 从 0x405e00 取函数地址并间接调用两次：

```
4025b0: and $0x38,%ebx
4025b3: and $0x7,%eax
4025b6: call *0x405e00(,%rax,8)
4025bd: xor $0x6d14,%eax
4025c4: call *0x405e00(%rbx)
4025cf: add $0x3b26,%eax
```

函数表的顺序是 [op5,op2,op1,op0,op2,op4,op3,op3]。六个函数的运算如下，rol32/ror32 分别表示 32 位循环左移和右移：

函数	运算
op0	rol32(x,3)^0xc2fa
op1	ror32(x,7)
op2	(x^0x3900)+4*x
op3	((x<<5)+0x3905)^(5*x)
op4	(x+0x8a65)^(x>>3)
op5	(x^(x>>3))+0x5a48

op2 的 xor $0x39,%ah 修改 eax 的第 8～15 位，因此对应 x^0x3900。总结果为：

```
x = u32(c>>6)^u32(c>>32)
i = c&7
j = (c>>3)&7
answer = table[j](table[i](x)^0x6d14)+0x3b26
```

第二次调用中的 c&0x38 已经是字节偏移，相当于 j*8。先进行第一次调用，再异或 0x6d14，然后进行第二次调用，最后加上 0x3b26。

Honor：相邻字段与校验

secret_gate 的复制起点是 rbp-0xd0，检查位置为 rbp-0xb0，传给 secret_phase 的字符串从 rbp-0xac 开始：

```
40274b: lea -0xd0(%rbp),%r12
402771: sub $0x24,%rax
402775: cmp $0x7f,%rax
402779: jbe 402790
402796: call secret_copy
40279b: cmpl $0x4e45504f,-0xb0(%rbp)
4027ad: lea -0xac(%rbp),%rdi
4027b4: call secret_phase
```

检查字段和整数文本距复制起点分别为 32、36 字节。按小端序，0x4e45504f 对应字符 OPEN。输入长度需要在 36～163 字节内，不计字符串结束符，因此构造为：

```
32 个 A + OPEN + selector + 空格 + second + 空格 + third
```

这样先覆盖检查字段，再让 secret_phase 读取后面的三个整数。这一行最多 59 个字符，加上末尾零字符也在对象可用空间内。

令 C 为 Honor challenge，T 为前六关口令经驱动的 transcript_context 生成的 64 位 context：

```
k = u32(T)&3
A = u32(C)^u32(C>>32)
B = u32(T)^u32(T>>32)
second = ror32(A^B,7)^0x23a3^(k*0x5805)^u32(T>>16)
third = second^(B+k+0x7ba6+(A>>7))
```

secret_mix 是循环右移 7 位后异或 0x23a3，代入后可得到这三个整数。context 由前六关的口令计算得到，代码直接使用传入的值。验证时将同一 seed 的六关口令和 Honor 输入依次送入同一次 bomb 运行。

验证结果

环境为 WSL Ubuntu、Linux x86_64，编译参数为 -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror。2026-10-30 重新编译并测试，正式六关公开测试 60/60、Honor 10/10 通过。

参考课程说明和模板，用 objdump、GDB 分析，用 GCC、Python 验证，Codex 辅助编写完整求解代码和报告。

另外进行了以下测试：

测试	结果	检查内容
第一关直接 challenge	204/204	低位、高位、单比特、随机输入
第二关直接 challenge	204/204	移位片段与递推
第三关直接 challenge	204/204	8 个 selector 全覆盖
第四关直接 challenge	332/332	128 个 target 全覆盖
第五关直接 challenge	236/236	32 个起点/步数组合全覆盖
第六关直接 challenge	268/268	64 对函数选择全覆盖
公开列表以外的完整六关＋Honor	128/128	相同 seed 的完整输入链
输出容量及边界检查	80955 项	cap 0～256、哨兵检查、ASan/UBSan 无报告

使用公开列表以外的 seed 20261029，六关与 Honor 在同一次运行中全部通过，结果如下：

关卡	challenge	输出口令
1	0x1ad27693d04a0d2f	2622955666
2	0x9dfa919569802e0c	8 44 243 1233 6180 30903
3	0x800a9ac407779fcc	4 18686928
4	0xc75a17cd7c745b40	3507382101
5	0x1e857b19b205738e	985186108 2352
6	0x9052ac07ff217f59	1932961906

Honor 第七行：

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN0 4253943975 3933565719
```

在仓库目录中执行：

```
make
python3 public_test.py
./defuser --seed 20261029 > /tmp/bomblab-submission-input.txt
./defuser --secret --seed 20261029 >> /tmp/bomblab-submission-input.txt
./bomb --seed 20261029 < /tmp/bomblab-submission-input.txt
```

下图为该 seed 的实际运行结果，截图日期为 2026-10-29，2026-10-30 复核通过：

